

Table 17-1: Numeric Dunder Methods

Dunder method	Operation	Operator or built-in function
<code>__add__()</code>	Addition	<code>+</code>
<code>__sub__()</code>	Subtraction	<code>-</code>
<code>__mul__()</code>	Multiplication	<code>*</code>
<code>__matmul__()</code>	Matrix multiplication (new in Python 3.5)	<code>@</code>
<code>__truediv__()</code>	Division	<code>/</code>
<code>__floordiv__()</code>	Integer division	<code>//</code>
<code>__mod__()</code>	Modulus	<code>%</code>
<code>__divmod__()</code>	Division and modulus	<code>divmod()</code>
<code>__pow__()</code>	Exponentiation	<code>**</code> , <code>pow()</code>
<code>__lshift__()</code>	Left shift	<code>>></code>
<code>__rshift__()</code>	Right shift	<code><<</code>
<code>__and__()</code>	Bitwise and	<code>&</code>
<code>__or__()</code>	Bitwise or	<code> </code>
<code>__xor__()</code>	Bitwise exclusive or	<code>^</code>
<code>__neg__()</code>	Negation	Unary <code>-</code> , as in <code>-42</code>
<code>__pos__()</code>	Identity	Unary <code>+</code> , as in <code>+42</code>
<code>__abs__()</code>	Absolute value	<code>abs()</code>
<code>__invert__()</code>	Bitwise inversion	<code>~</code>
<code>__complex__()</code>	Complex number form	<code>complex()</code>
<code>__int__()</code>	Integer number form	<code>int()</code>
<code>__float__()</code>	Floating-point number form	<code>float()</code>
<code>__bool__()</code>	Boolean form	<code>bool()</code>
<code>__round__()</code>	Rounding	<code>round()</code>
<code>__trunc__()</code>	Truncation	<code>math.trunc()</code>
<code>__floor__()</code>	Rounding down	<code>math.floor()</code>
<code>__ceil__()</code>	Rounding up	<code>math.ceil()</code>

Some of these methods are relevant to our `wizCoin` class. Try writing your own implementation of the `__sub__()`, `__pow__()`, `__int__()`, `__float__()`, and `__bool__()` methods. You can see an example of an implementation at <https://autbor.com/wizcoinfull>. The full documentation for the numeric dunder methods is in the Python documentation at